MSC2050 Discrete Mathematics, Week 13

Dr. Anna Tomskova

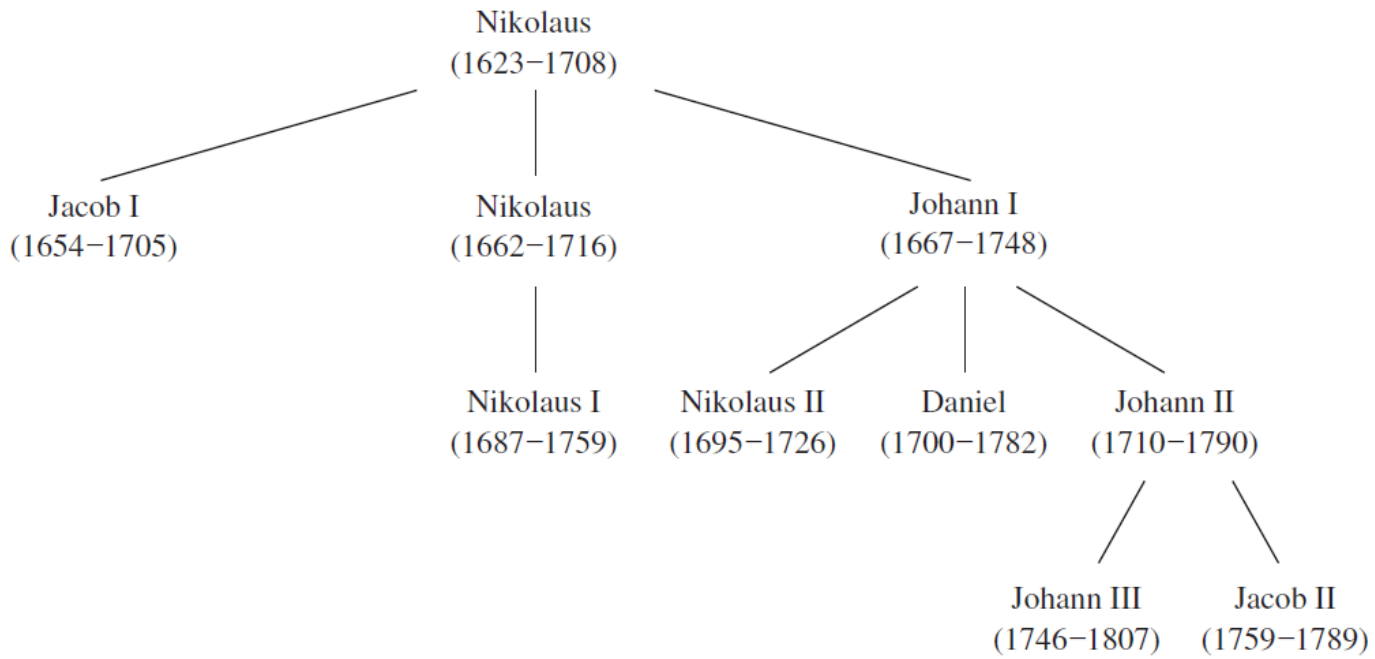
Inha University in Tashkent

Spring 2024
Version 1.0 typed under LaTeX

Introduction to trees

(Paragraph 11.1)

The undirected graph representing a family tree is an example of a tree.



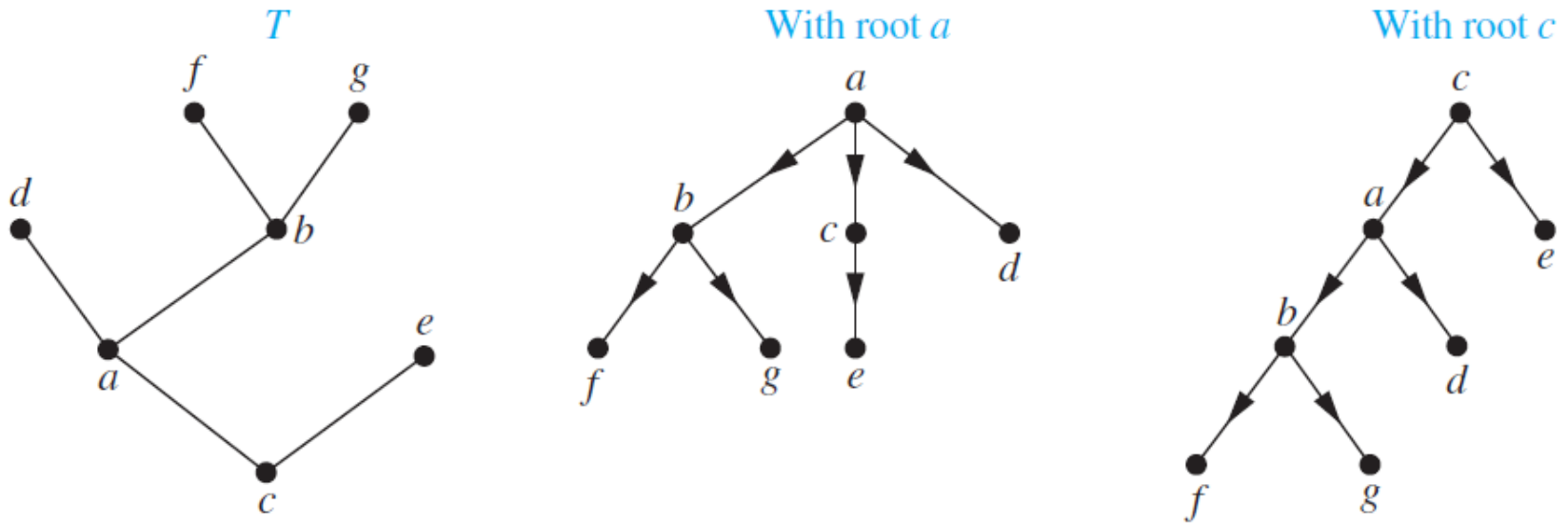
A tree cannot have:

- a simple circuit
- multiple edges
- loops
- isolated points

A tree is a connected undirected graph with no simple circuits.

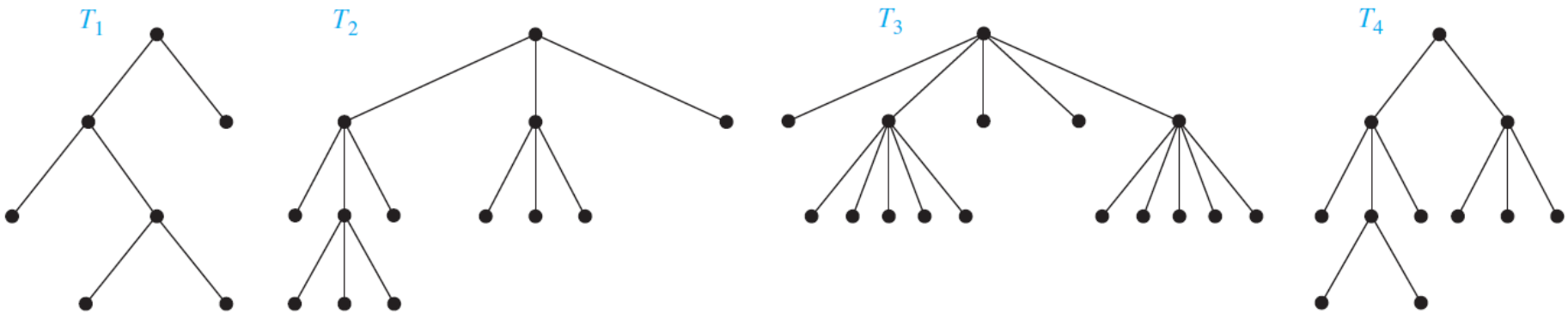
Rooted trees

A rooted tree is a tree in which one vertex has been designated as the root and every edge is directed away from the root.



The parent of v is the unique vertex u s. t. there is a directed edge from u to v. When u is the parent of v, v is called a child of u. Vertices with the same parent are called siblings. The ancestors are the vertices in the path from the root to vertex, excluding the vertex itself and including the root. The descendants of a vertex v are those vertices that have v as an ancestor. A vertex is called a leaf if it has no children. Vertices that have children are called internal vertices.

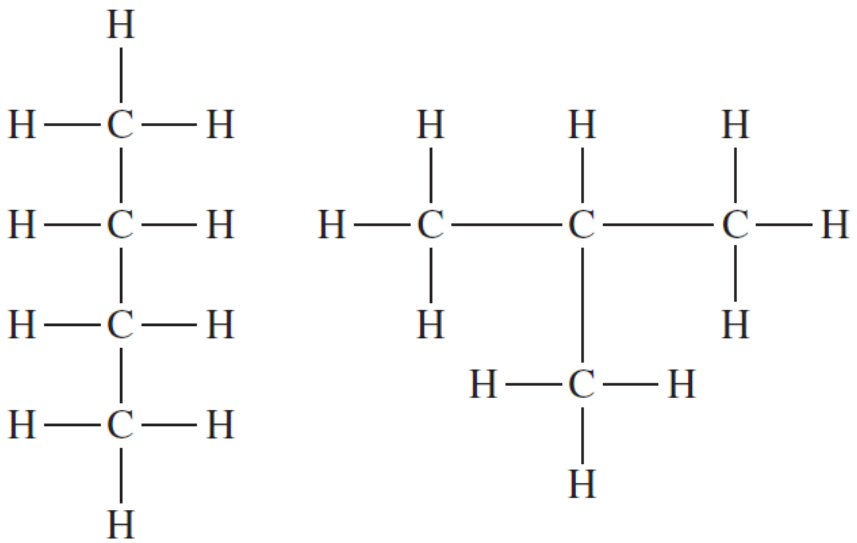
A rooted tree is called an m-ary tree if every internal vertex has no more than m children.
The tree is called a full m-ary tree if every internal vertex has exactly m children.
An m-ary tree with $m = 2$ is called a binary tree.



T_1 is a full binary tree.
T_2 is a full 3-ary tree.
T_3 is a full 5-ary tree.
T_4 is a 3-ary tree.

Saturated Hydrocarbons

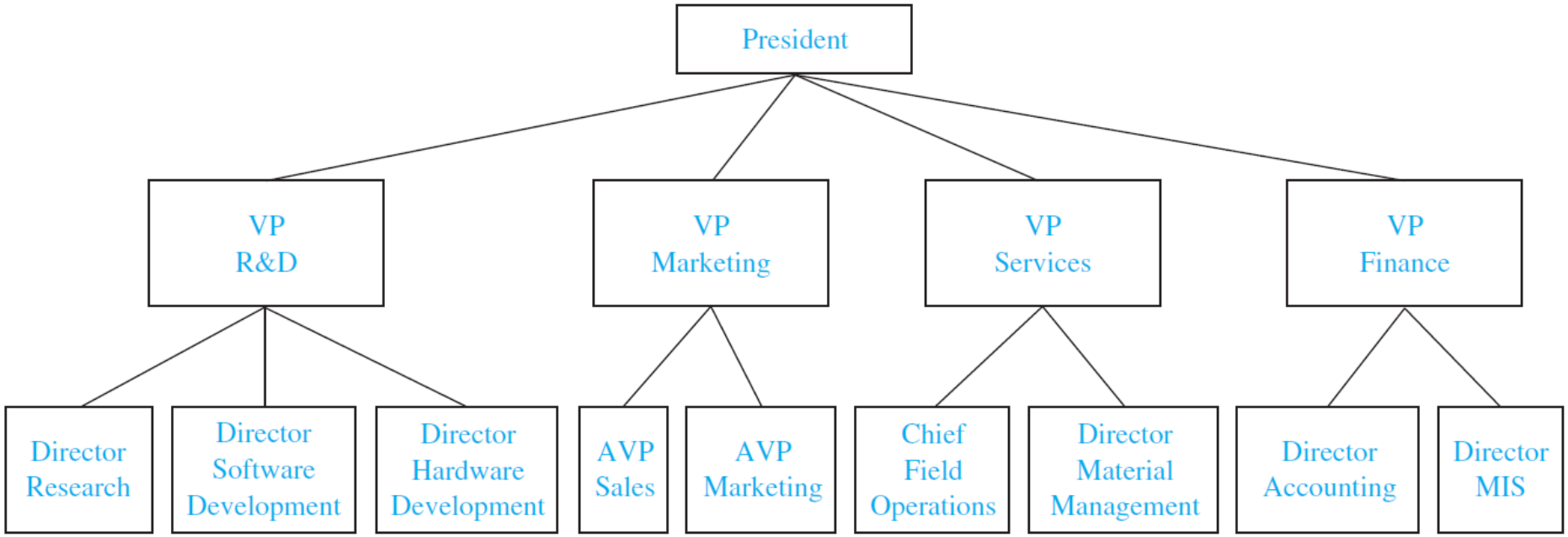
Trees can be used to represent molecules, where atoms are represented by vertices and bonds between them by edges.
The English mathematician Arthur Cayley discovered trees in 1857 when he was trying to enumerate the isomers of compounds of the form C_nH_{2n+2}, which are called saturated hydrocarbons.



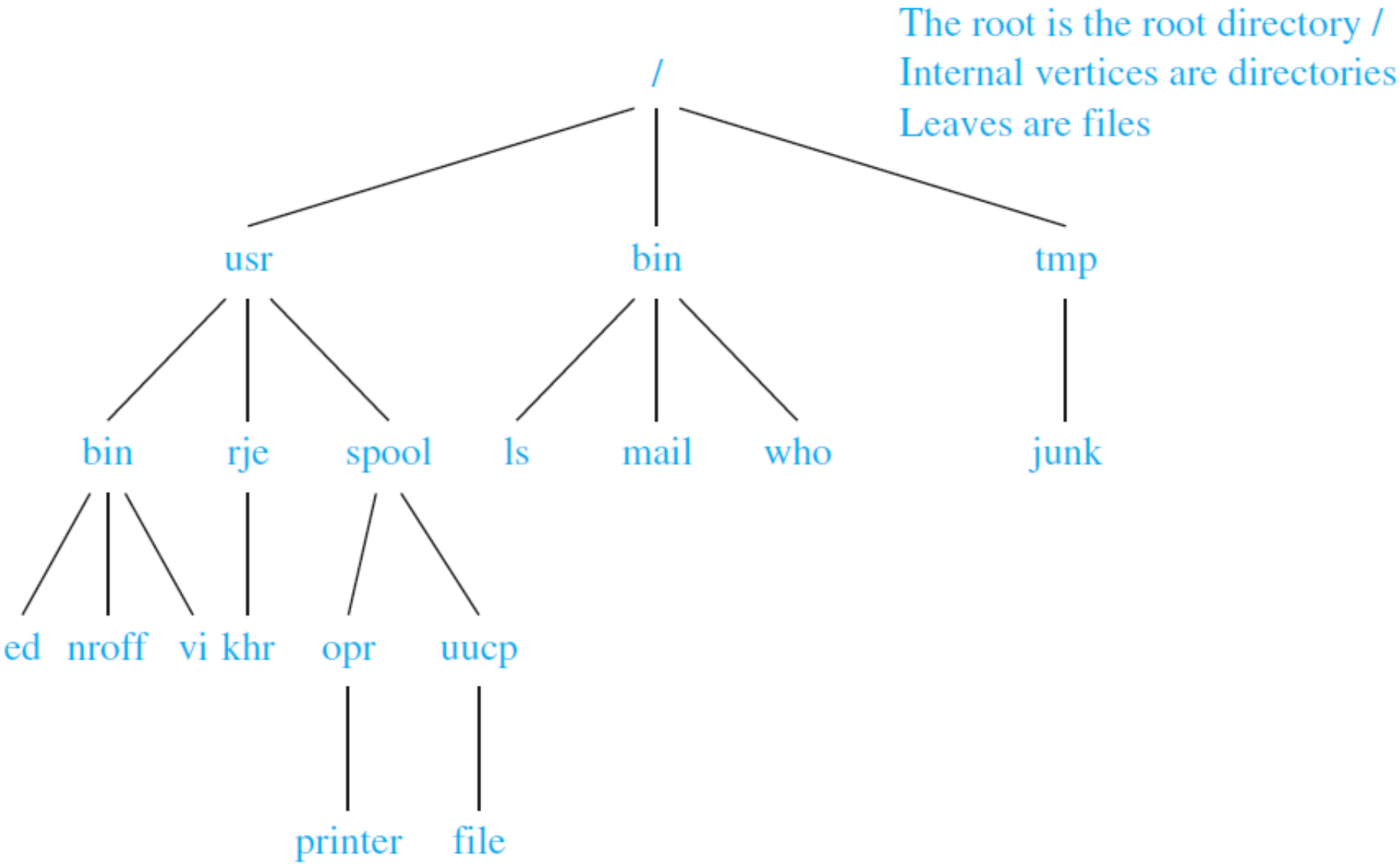
Butane Isobutane

The non-isomorphic trees with n vertices of degree 4 and $2n + 2$ of degree 1 represent the different isomers of C_nH_{2n+2}. For instance, when $n = 4$, there are exactly two non-isomorphic trees of this type.

Representing Organizations

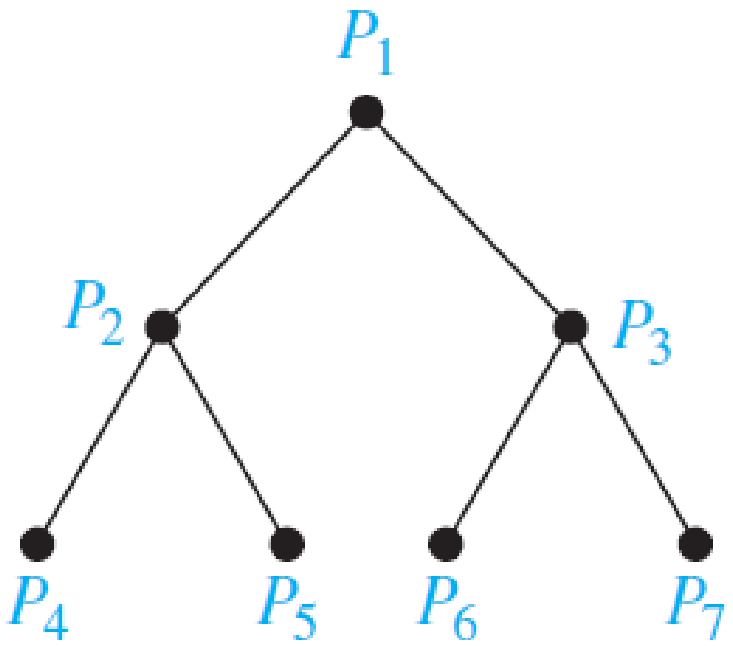


Computer File Systems



Parallel Processors

A tree-connected network is an important way to interconnect processors.
The graph representing such a network is a binary tree.
Such a network interconnects $n = 2^k - 1$ processors.



Properties of trees

Theorem

A tree with n vertices has $n - 1$ edges.

Proof: We will use mathematical induction to prove this theorem.
BASIS STEP: When $n = 1$, a tree with $n = 1$ vertex has no edges. It follows that the theorem is true for $n = 1$.
INDUCTIVE STEP: The inductive hypothesis states that every tree with k vertices has $k - 1$ edges.
Suppose that a tree T has $k + 1$ vertices and that v is a leaf of T, and let w be the parent of v.
Removing from T the vertex v and the edge connecting w to v produces a tree with k vertices, because the resulting graph is still connected and has no simple circuits. By the inductive hypothesis, it has $k - 1$ edges.
It follows that T has k edges because it has one more edge, the edge connecting v and w. This completes the inductive step.

Theorem

A full m-ary tree with i internal vertices contains $n = mi + 1$ vertices.

Proof: Every vertex, except the root, is the child of an internal vertex. Because each of the i internal vertices has m children, there are mi vertices in the tree other than the root. Therefore, the tree contains $n = mi + 1$ vertices.

Theorem

A full m-ary tree with
(i) n vertices has $i = (n-1)/m$ internal vertices and $l = [(m-1)n+1]/m$ leaves,
(ii) i internal vertices has $n = mi + 1$ vertices and $l = (m-1)i + 1$ leaves,
(iii) l leaves has $n = (ml-1)/(m-1)$ vertices and $i = (l-1)/(m-1)$ internal vertices.

Proof: Let n represent the number of vertices, i the number of internal vertices, and l the number of leaves. The three parts of the theorem can all be proved using the equality $n = mi + 1$, together with the equality $n = l + i$, which is true because each vertex is either a leaf or an internal vertex. We will prove part (i) here. The proofs of parts (ii) and (iii) are similar.
Solving for i in $n = mi + 1$ gives $i = (n-1)/m$. Then inserting this expression for i into the equation $n = l + i$ shows that

$$l = n - i = n - (n-1)/m = [(m-1)n + 1]/m.$$

Example

Suppose that someone starts a chain letter. Each person who receives the letter is asked to send it on to four other people. Some people do this, but others do not send any letters. How many people have seen the letter, including the first person, if no one receives more than one letter and if the chain letter ends after there have been 100 people who read it but did not send it out? How many people sent out the letter?

Answers: $n = 133$, $i = 33$

Answer Q1

Homework:

- Solve odd exercises # 1-37 at p. 755.

Tree traversal

(Paragraph 11.3)

Traversal is a process to visit all the nodes of a tree and may use their values. There are three ways which we use to traverse a tree:

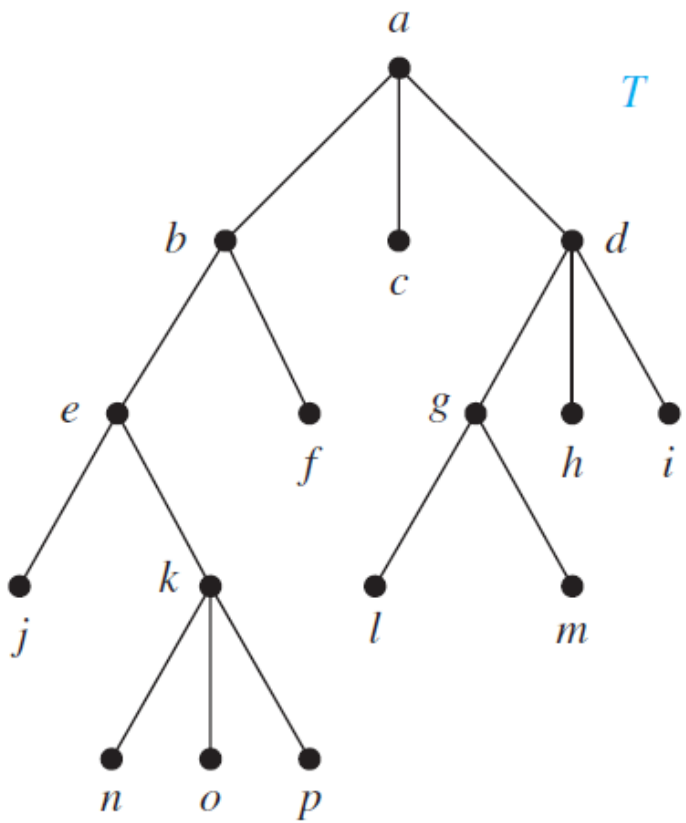
- Pre-order Traversal
- In-order Traversal
- Post-order Traversal

Pre-order Traversal:
Step 1- Visit root node.
Step 2- Recursively traverse left subtree.
Step 3- Recursively traverse right subtree.

In-order Traversal:
Step 1- Recursively traverse left subtree.
Step 2- Visit root node.
Step 3- Recursively traverse right subtree.

Post-order Traversal:
Step 1- Recursively traverse left subtree.
Step 2- Recursively traverse right subtree.
Step 3- Visit root node.

Pre-order Traversal

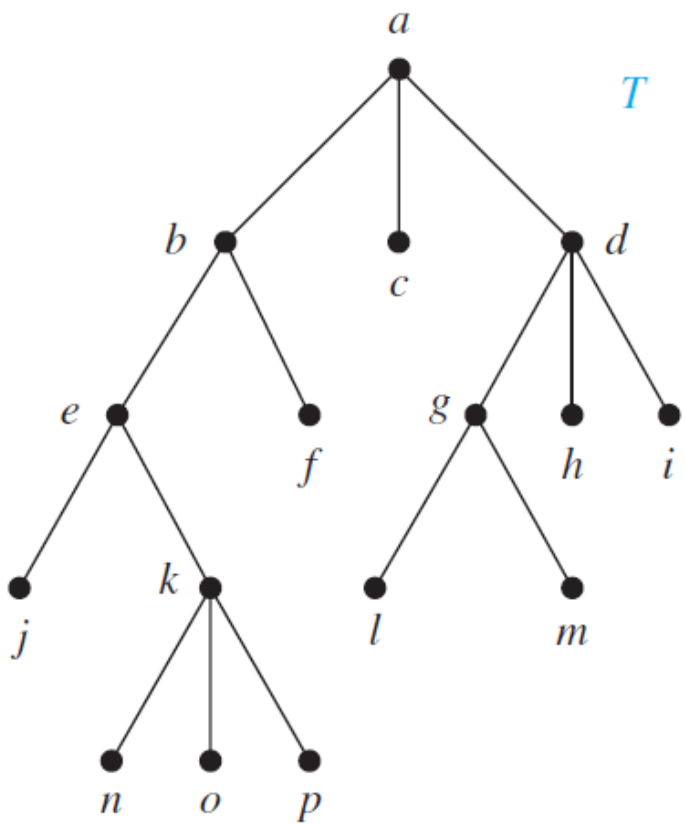


$$a, b, e, j, k, n, o, p, f, c, d, g, l, m, h, i$$

Algorithm

procedure preorder(T : rooted tree)
$r :=$ root of T
list r
for each child c of r from left to right
$T(c) :=$ subtree with c as its root
preorder($T(c)$)

In-order Traversal

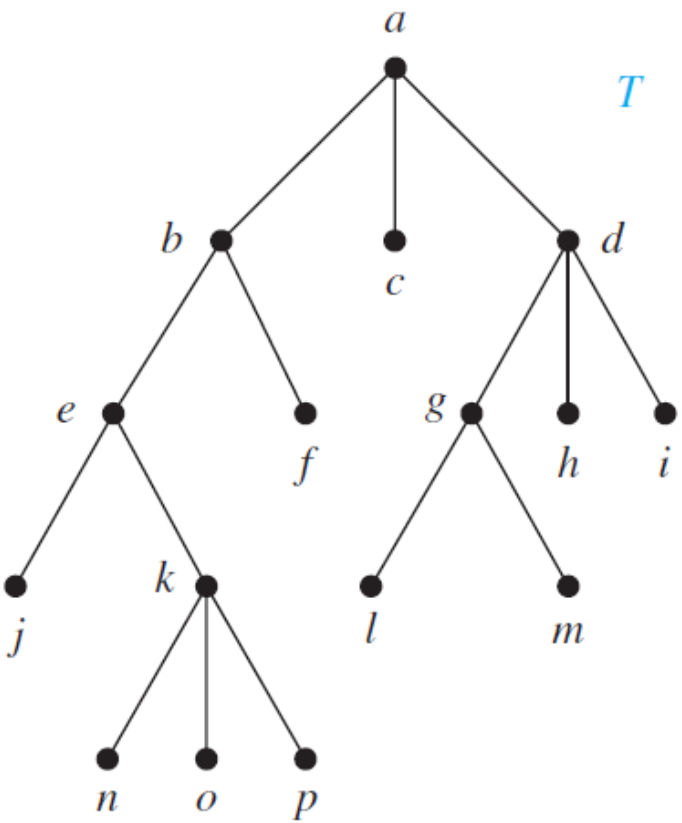


$j, e, n, k, o, p, b, f, a, c, l, g, m, d, h, i$

Algorithm

procedure inorder(T : rooted tree)
r :=root of T
if r is a leaf then list r
else l :=first child of r from left to right
    $T(l)$:= subtree with l as its root
    inorder$(T(l))$
    list r
    for each child c of r except for l from left to right
        $T(c)$:= subtree with c as its root
        inorder$(T(c))$

Post-order Traversal

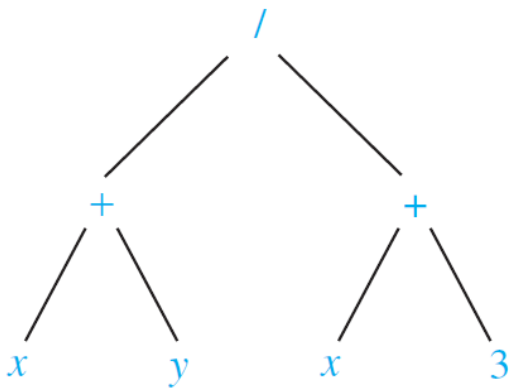


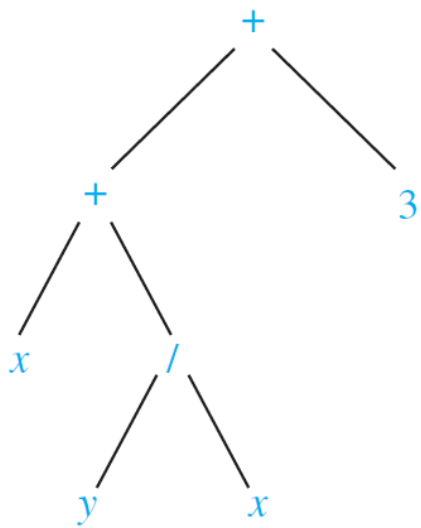
$$j, n, o, p, k, e, f, b, c, l, m, g, h, i, d, a$$

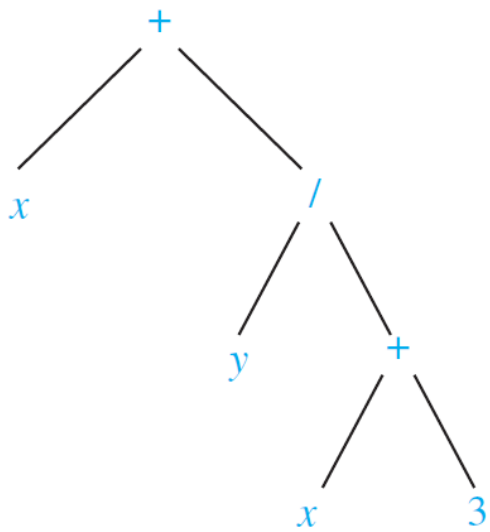
Algorithm

procedure postorder(T : rooted tree)
$r :=$ root of T
for each child c of r from left to right
    $T(c) :=$ subtree with c as its root
    postorder($T(c)$)
list r

Infix form can be ambiguous







In-order traversal (infix form) gives: $x + y/x + 3$
It is necessary to include parentheses.

The prefix form (Polish notation) is $/ + xy + x3 \quad + +x/yx3 \quad + x/y + x3$

The postfix form (reverse Polish notation) is
$xy + x3 + / \quad xyx/ + 3 + \quad xyx3 + /+$

Both prefix and postfix forms are unambiguous and parentheses are not needed.

Evaluating prefix and postfix expressions

What is the value of the prefix expression $+ - *235/ \uparrow 234$?
$\uparrow$ is taking a power
Answer: 3

What is the value of the postfix expression $723 * -4 \uparrow 93/+$?
Answer: 4

Answer Q2

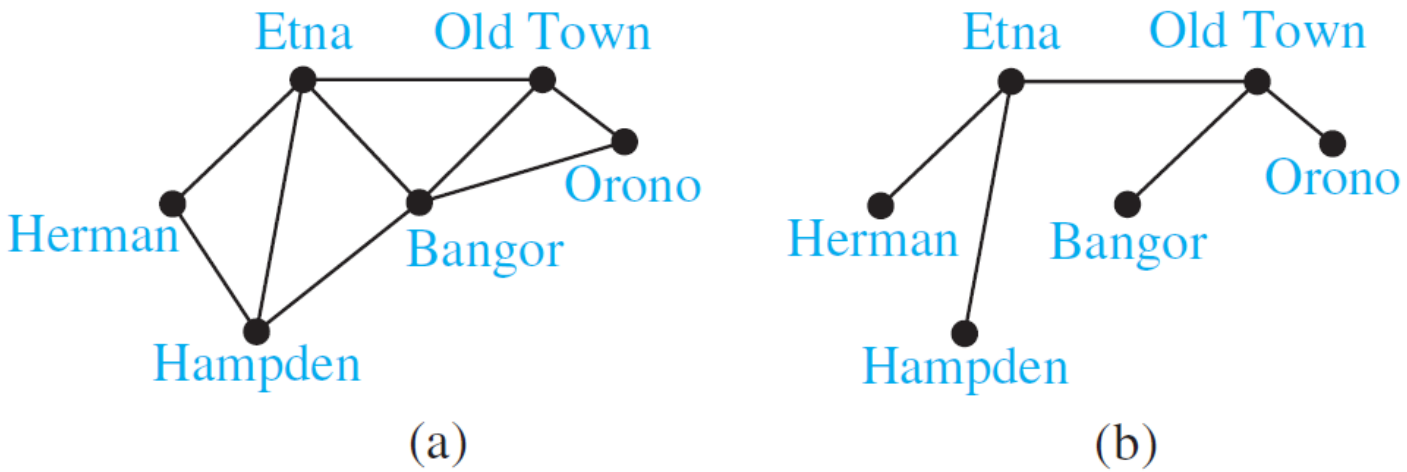
Homework:

- Solve odd exercises starting at p. 783.

Spanning trees

(Paragraph 11.4)

Let G be a simple graph. A spanning tree of G is a subgraph of G that is a tree containing every vertex of G.



(a) is a road system
(b) At least five roads must be plowed to ensure that there is a path between any two towns.

Theorem

A simple graph is connected if and only if it has a spanning tree.

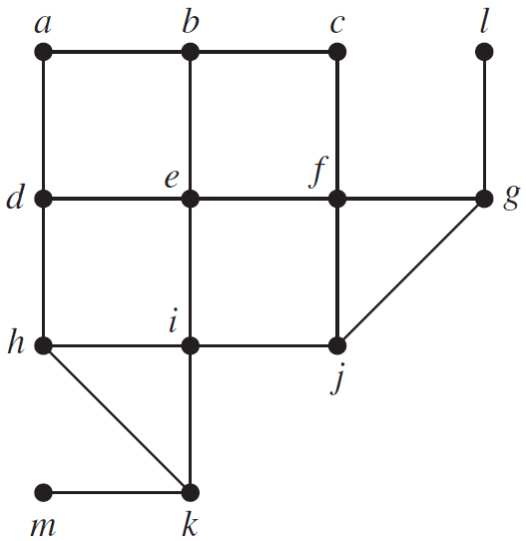
Proof: First, suppose that a simple graph G has a spanning tree T. T contains every vertex of G. Furthermore, there is a path in T between any two of its vertices. Because T is a subgraph of G, there is a path in G between any two of its vertices. Hence, G is connected.

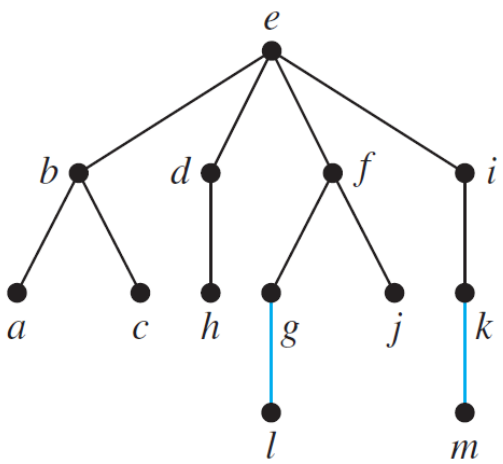
Now suppose that G is connected. If G is not a tree, it must contain a simple circuit. Remove an edge from one of these simple circuits. The resulting subgraph has one fewer edge but still contains all the vertices of G and is connected.
If this subgraph is not a tree, it has a simple circuit; so as before, remove an edge that is in a simple circuit. Repeat this process until no simple circuits remain. The process terminates when no simple circuits remain. A tree is produced because the graph stays connected as edges are removed. This tree is a spanning tree because it contains every vertex of G.

Breadth-First search

Algorithm: Breadth-First Search

procedure BFS (G : connected graph with vertices v_1, v_2, ... , v_n)
$T :=$ tree consisting only of vertex v_1
$L :=$ empty list
put v_1 in the list L of unprocessed vertices
while L is not empty
    remove the first vertex, v, from L
    for each neighbor w of v
        if w is not in L and not in T **then**
            add w to the end of the list L, add w and edge (v, w) to T





The complexity is $O(n^2)$.

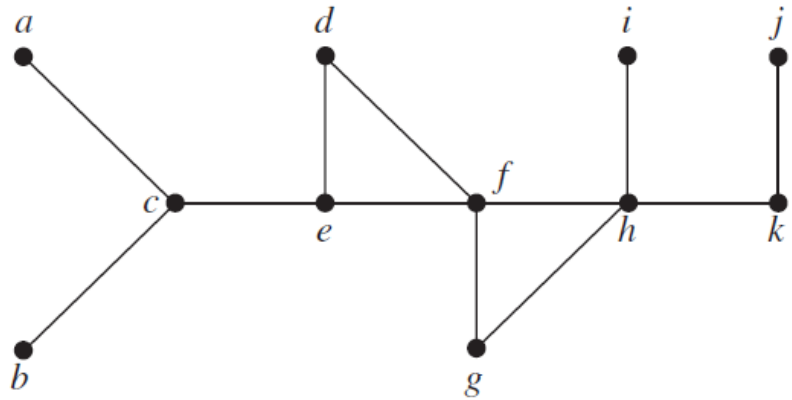
Depth-First search

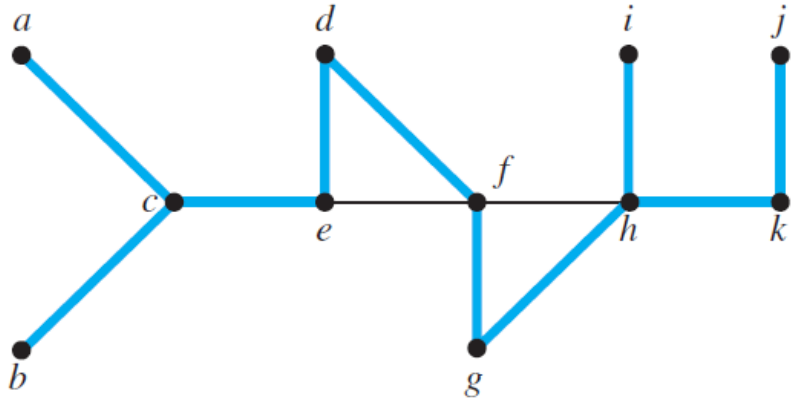
This is a backtracking algorithm

Algorithm: Depth-First Search

procedure DFS(G: connected graph with vertices $v_1, v_2, \ldots, v_n$)
$T :=$ tree consisting only of the vertex v_1
visit(v_1)

procedure visit(v: vertex of G)
for each vertex w adjacent to v and not yet in T add vertex w, edge (v, w) to T
    visit(w)





The complexity is $O(n^2)$.

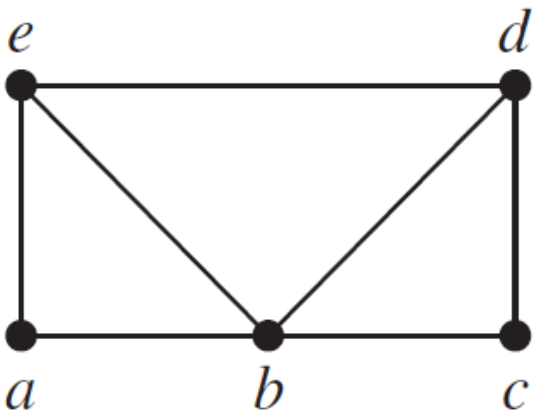
Answer Q3

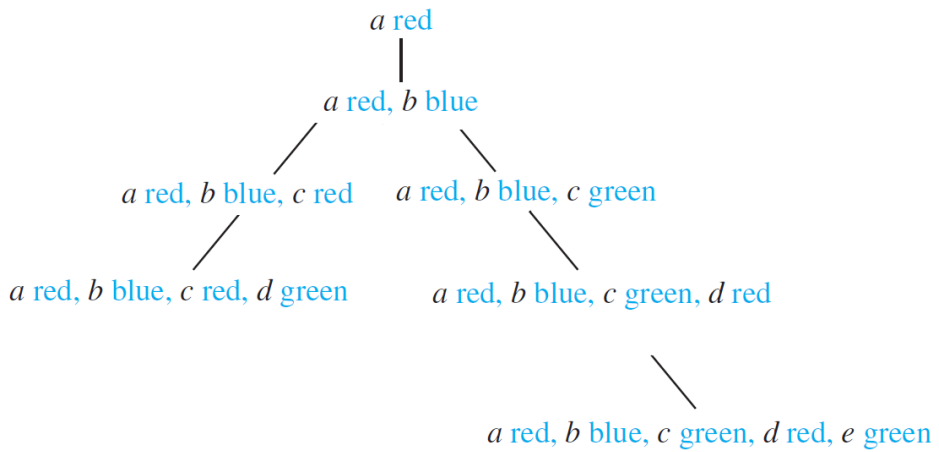
Backtracking applications

Some problems can be solved only by performing an exhaustive search of all possible solutions. One way to search systematically for a solution is to use a decision tree, where each internal vertex represents a decision and each leaf a possible solution.

To find a solution via backtracking, first make a sequence of decisions in an attempt to reach a solution as long as this is possible. The sequence of decisions can be represented by a path in the decision tree. Once it is known that no solution can result from any further sequence of decisions, backtrack to the parent of the current vertex and work toward a solution with another series of decisions, if this is possible. The procedure continues until a solution is found, or it is established that no solution exists.

Graph Colorings: Backtracking to decide whether a graph can be colored using 3 colors?

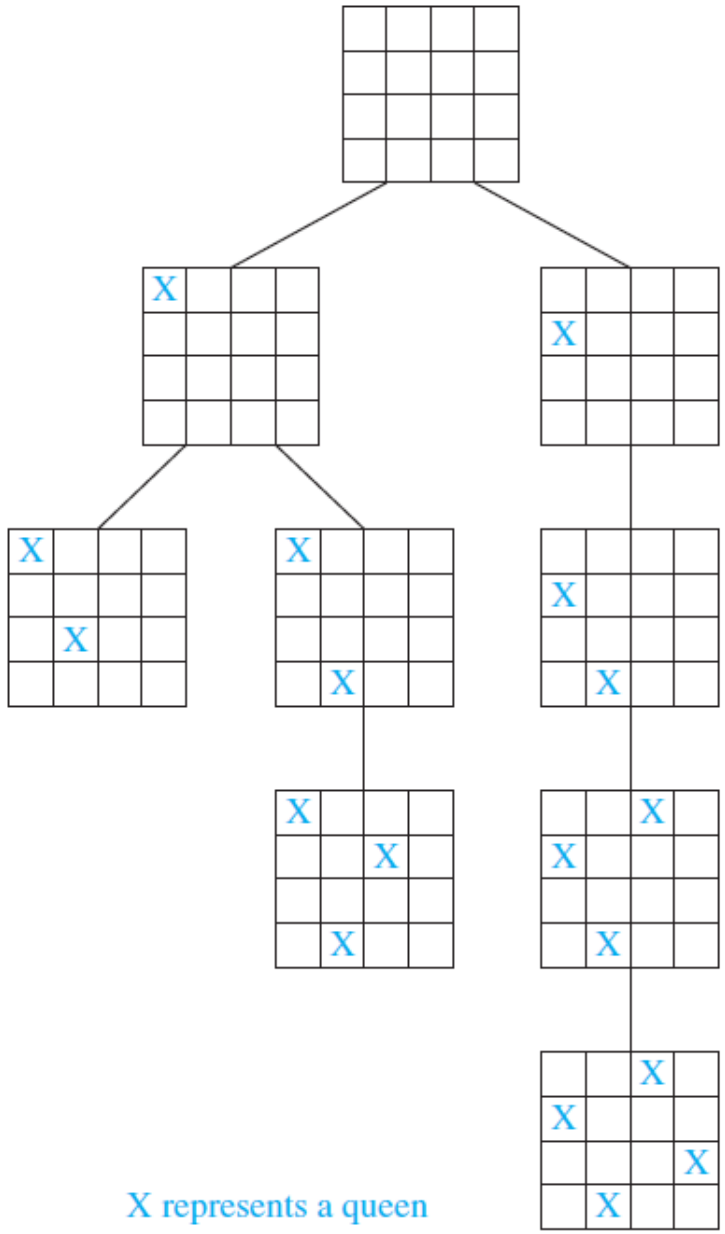




The n-Queens problem

The n-queens problem asks how n queens can be placed on an $n \times n$ chessboard so that no two queens can attack one another.
How can backtracking be used to solve this problem?

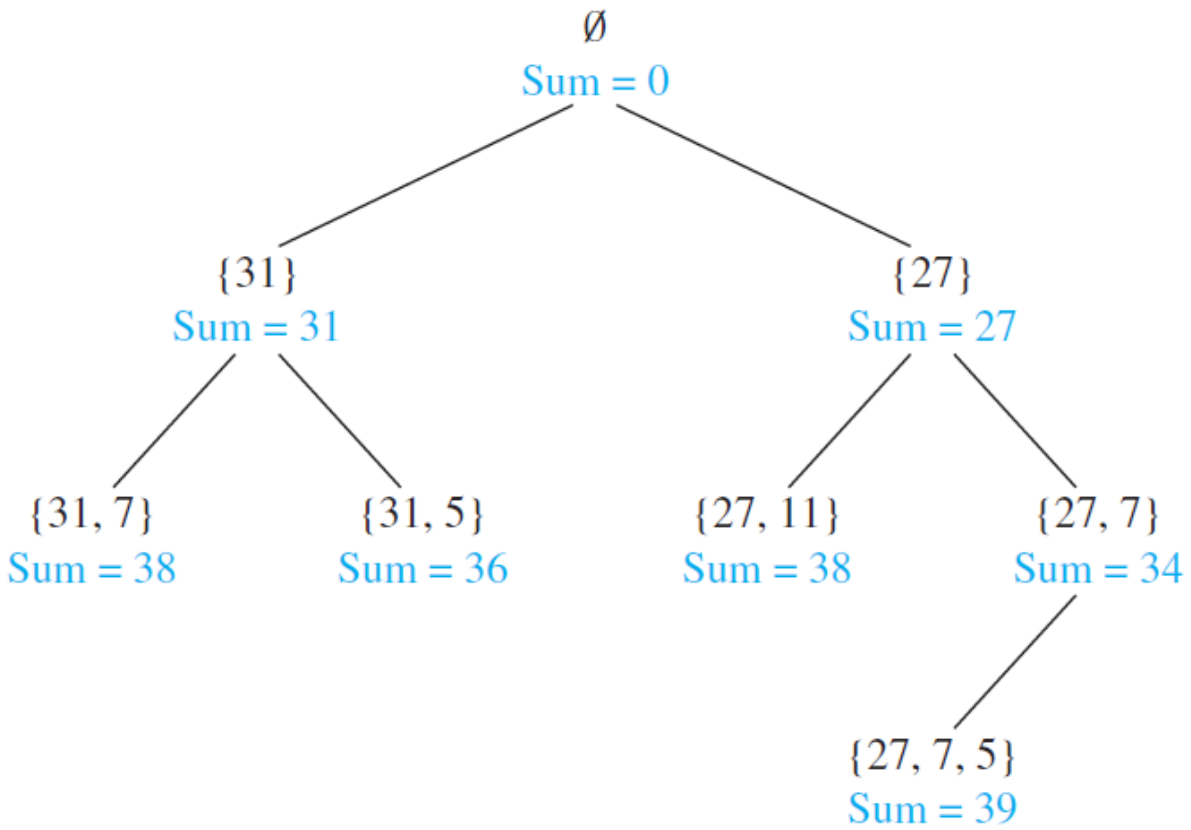
For example take $n = 4$ we have



Sum of subsets

Given a set of positive integers $x_1, x_2, \ldots, x_n$, find a subset of this set of integers that has M as its sum. How can backtracking be used to solve this problem?

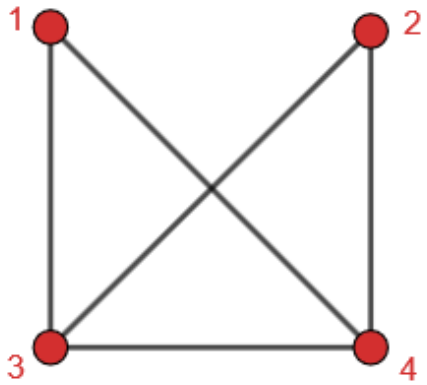
For example, find a subset of $\{31, 27, 15, 11, 7, 5\}$ with the sum equal to 39.

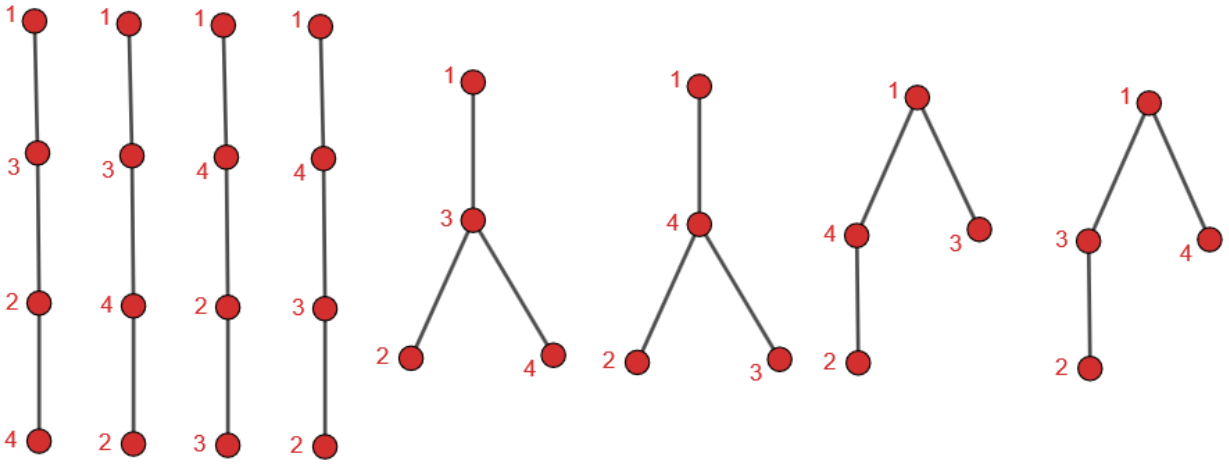


Total number of spanning trees

How many spanning trees has a simple graph? How can backtracking be used to solve this problem?

For example, consider a graph





Answer:

Number of spanning trees

Algorithm from here

Follow the given procedure :-

STEP 1: Create Adjacency Matrix for the given graph.

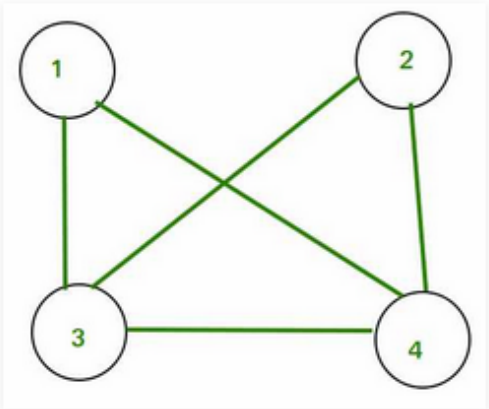
STEP 2: Replace all the diagonal elements with the degree of nodes. For eg. element at (1,1) position of adjacency matrix will be replaced by the degree of node 1, element at (2,2) position of adjacency matrix will be replaced by the degree of node 2, and so on.

STEP 3: Replace all non-diagonal 1's with -1.

STEP 4: Calculate co-factor for any element.

STEP 5: The cofactor that you get is the total number of spanning tree for that graph.

Consider the following graph:

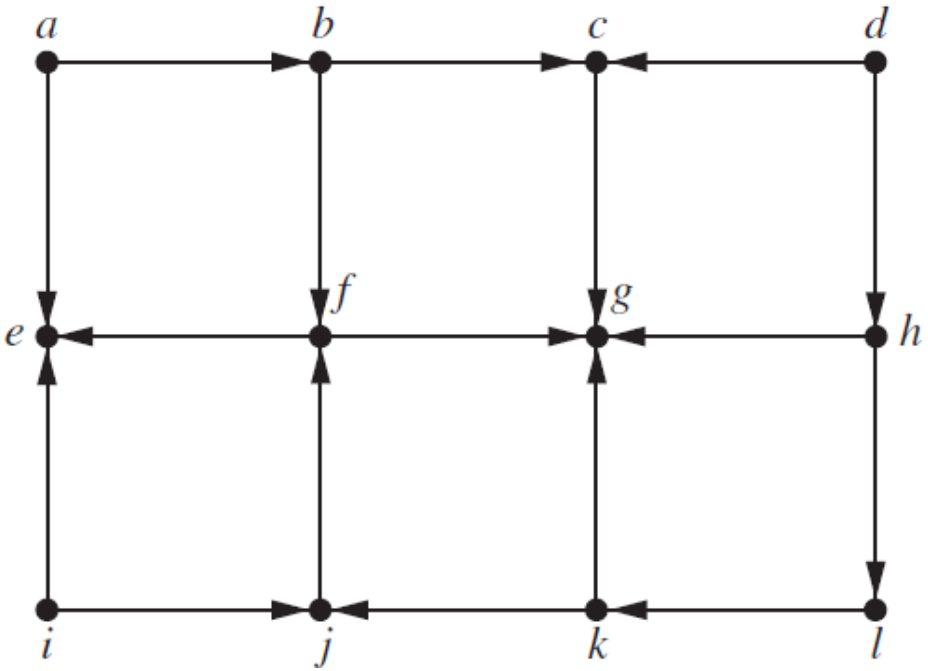


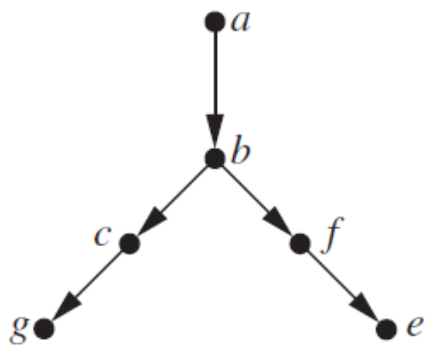
Answer Q4

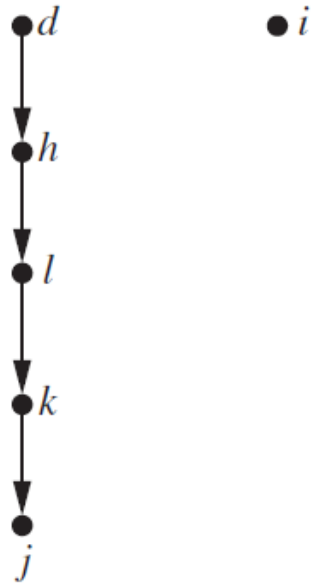
Depth-First search in directed graphs

We can easily modify both depth-first search and breadth-first search so that they can run given a directed graph as input.

- The output will not necessarily be a spanning tree, but rather a spanning forest.
- In both algorithms we add an edge only when it is directed away from the vertex.
- The next vertex added by the algorithm becomes the root of a new tree in the spanning forest.







Web spiders

The search engines (Google, Yahoo, etc.) use programs called Web spiders (or crawlers or bots) to visit websites and analyze their contents.
The World Wide Web can be modeled as a directed graph where each Web page is represented by a vertex and where an edge starts at the Web page a and ends at the Web page b if there is a link on a pointing to b.
Web spiders use both depth-first searching and breadth-first searching.

Using depth-first search, an initial Web page is selected, a link is followed to a second Web page (if there is such a link), a link on the second Web page is followed to a third Web page, if there is such a link, and so on, until a page with no new links is found. Backtracking is then used to examine links at the previous level to look for new links, and so on.

Using breadth-first search, an initial Web page is selected and a link on this page is followed to a second Web page, then a second link on the initial page is followed (if it exists), and so on, until all links of the initial page have been followed.

Homework:

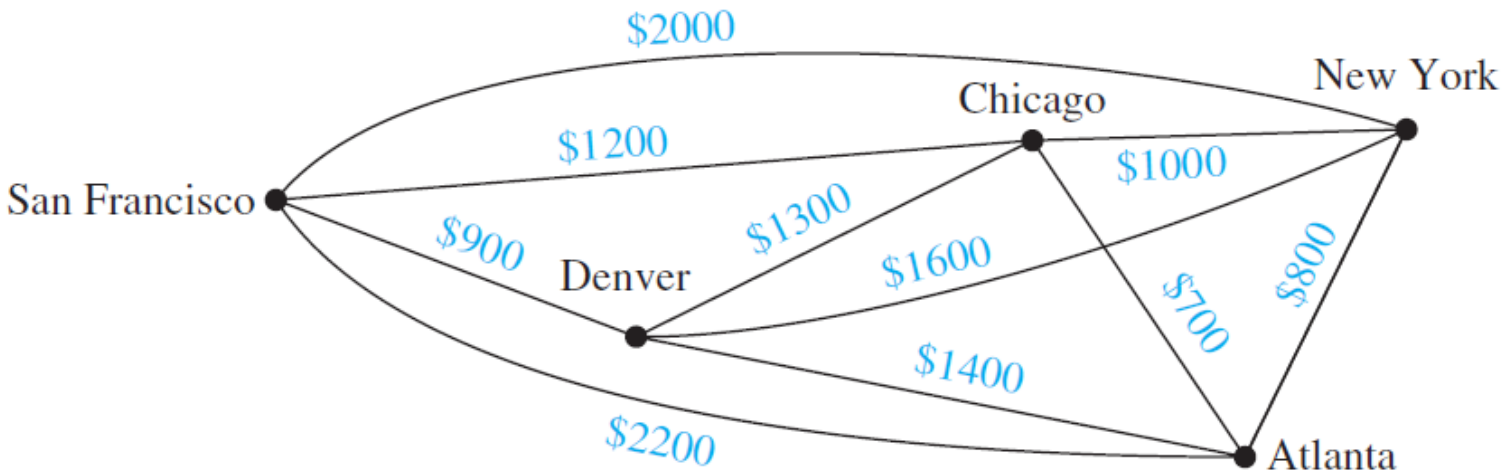
- Solve odd exercises until #29 starting at p. 795.

Minimum spanning trees

(Paragraph 11.5)

A graph is called weighted if there is a weight (cost) for each edge.

A weighted graph showing monthly lease costs for lines in a computer network



A minimum spanning tree in a connected weighted graph is a spanning tree that has the smallest possible sum of weights of its edges.

There are two algorithms Prim's algorithm and Kruskal's algorithm for finding minimum spanning tree.

Prim's algorithm

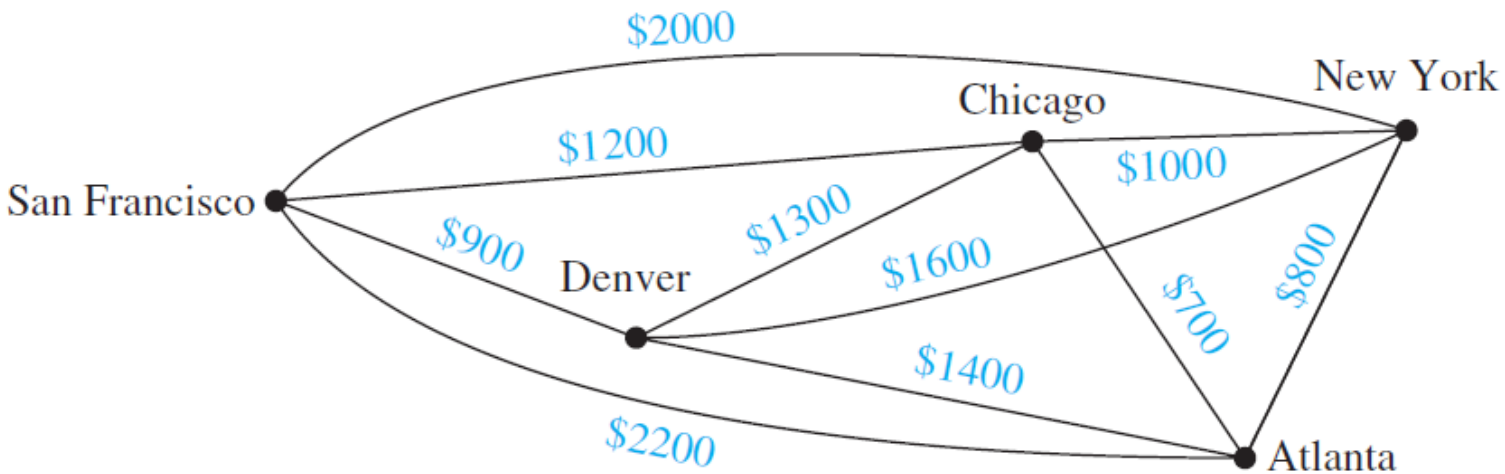
The first algorithm that we will discuss was originally discovered by the Czech mathematician Vojtěch Jarník in 1930, who described it in a paper in an obscure Czech journal.
The algorithm became well known when it was rediscovered in 1957 by Robert Prim.
Because of this, it is known as Primâs algorithm (and sometimes as the Prim-Jarník algorithm).

- Begin by choosing any edge with smallest weight, putting it into the spanning tree.
- Add to the tree edges of minimum weight that are incident to a vertex already in the tree.
- Never forming a simple circuit with those edges already in the tree.
- Stop when $n-1$ edges have been added.

Prim's Algorithm

procedure Prim(G : weighted connected undirected graph with n vertices)
$T :=$ a minimum-weight edge
for $i := 1$ **to** $n - 2$
    $e :=$ an edge of minimum weight incident to a vertex in T and not forming a simple circuit in T if added to T.
    $T := T$ with e added
return T (T is a minimum spanning tree of G)

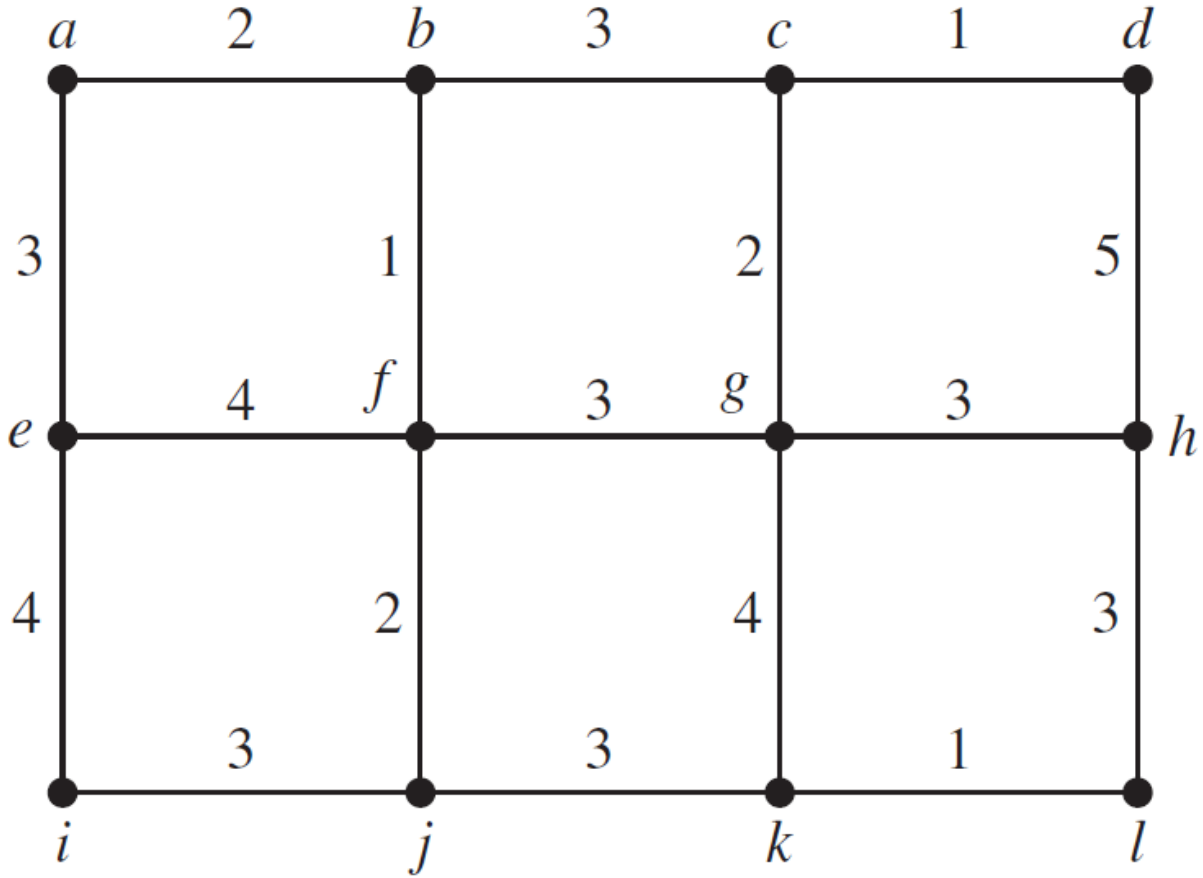
Find a minimum spanning tree using Prim's algorithm of the weighted graph



Answer: (Chicago, Atlanta), (Atlanta, New York) (Chicago, San Francisco), (San Francisco, Denver) with cost:700+800+1200+900=3600.

Example

Find a minimum spanning tree using Prim's algorithm of the weighted graph. What is the minimal cost?



Answer: 24

Answer Q5

Theorem

Prim's algorithm produces a minimum spanning tree of a connected weighted graph.

Proof: Suppose that the successive edges chosen by Prim's algorithm are $e_1, e_2, \ldots, e_{n-1}$ and S be the tree.
Let T be the minimum spanning tree of G.
The theorem follows if we can show that $S = T$.
Let k be a maximal integer such that S and T has k common edges.
We need to prove that $k = n - 1$.
Let $k < n - 1$, that is T contains $e_1, e_2, \ldots, e_k$, but not e_{k+1}.
Consider the graph made up of T together with e_{k+1}. Because this graph is connected and has n edges, too many edges to be a tree, it must contain a simple circuit. This simple circuit must contain e_{k+1} because there was no simple circuit in T.
Furthermore, if S_{k+1} is a tree with $e_1, e_2, \ldots, e_{k+1}$ as its edges, there must be an edge in the simple circuit that does not belong to S_{k+1} because S_{k+1} is a tree.

Proof(continuation): By starting at an endpoint of e_{k+1} that is also an endpoint of one of the edges $e_1, \ldots, e_k$, and following the circuit until it reaches an edge not in S_{k+1}, we can find an edge e not in S_{k+1} that has an endpoint that is also an endpoint of one of the edges $e_1, e_2, \ldots, e_k$.
By deleting e from T and adding e_{k+1}, we obtain a tree T' with $n-1$ edges (it is a tree because it has no simple circuits).
Note that the tree T' contains $e_1, e_2, \ldots, e_k, e_{k+1}$.
Furthermore, because e_{k+1} was chosen by Prim's algorithm at the $(k+1)$st step, and e was also available at that step, the weight of e_{k+1} is less than or equal to the weight of e. From this observation, it follows that T' is also a minimum spanning tree, because the sum of the weights of its edges does not exceed the sum of the weights of the edges of T. This contradicts the choice of k as the maximum integer such that a minimum spanning tree exists containing $e_1, \ldots, e_k$.
Hence, $k = n-1$, and $S = T$. It follows that Prim's algorithm produces a minimum spanning tree.

Kruskal's algorithm

The second algorithm we will discuss was discovered by Joseph Kruskal in 1956.

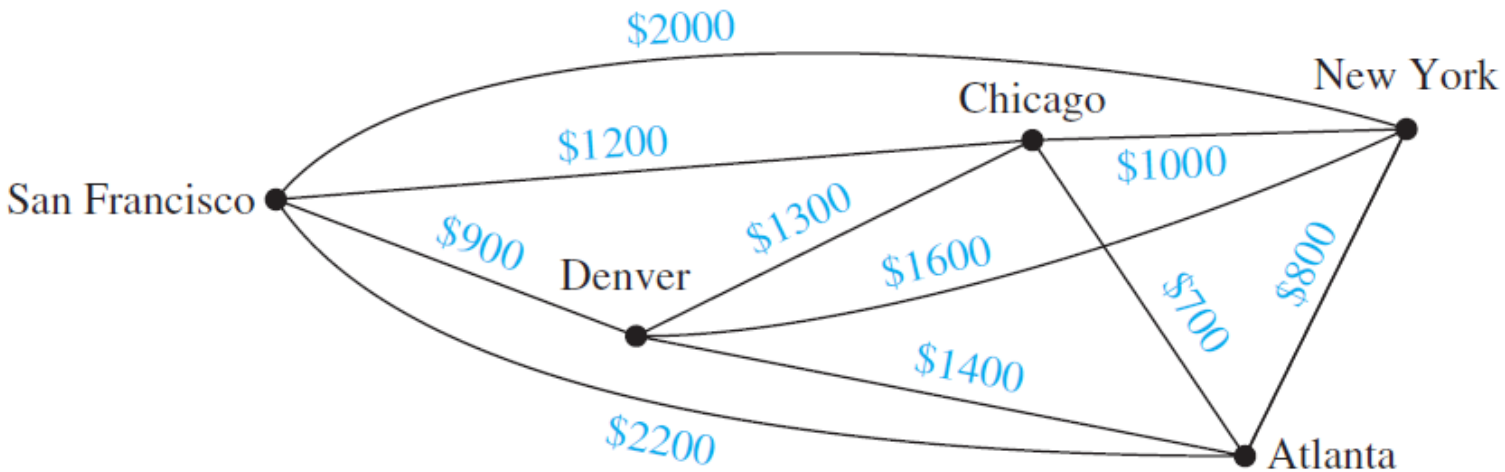
- Choose an edge in the graph with minimum weight
- add edges with minimum weight that do not form a simple circuit with those edges already chosen
- Stop after $n1$ edges have been selected

Kruskal's Algorithm

procedure Kruskal(G : weighted connected undirected graph with n vertices)
$T :=$ empty graph
for $i := 1$ **to** $n - 1$
    $e :=$ an edge with smallest weight not forming a simple circuit in T if added to T.
    $T := T$ with e added
return T (T is a minimum spanning tree of G)

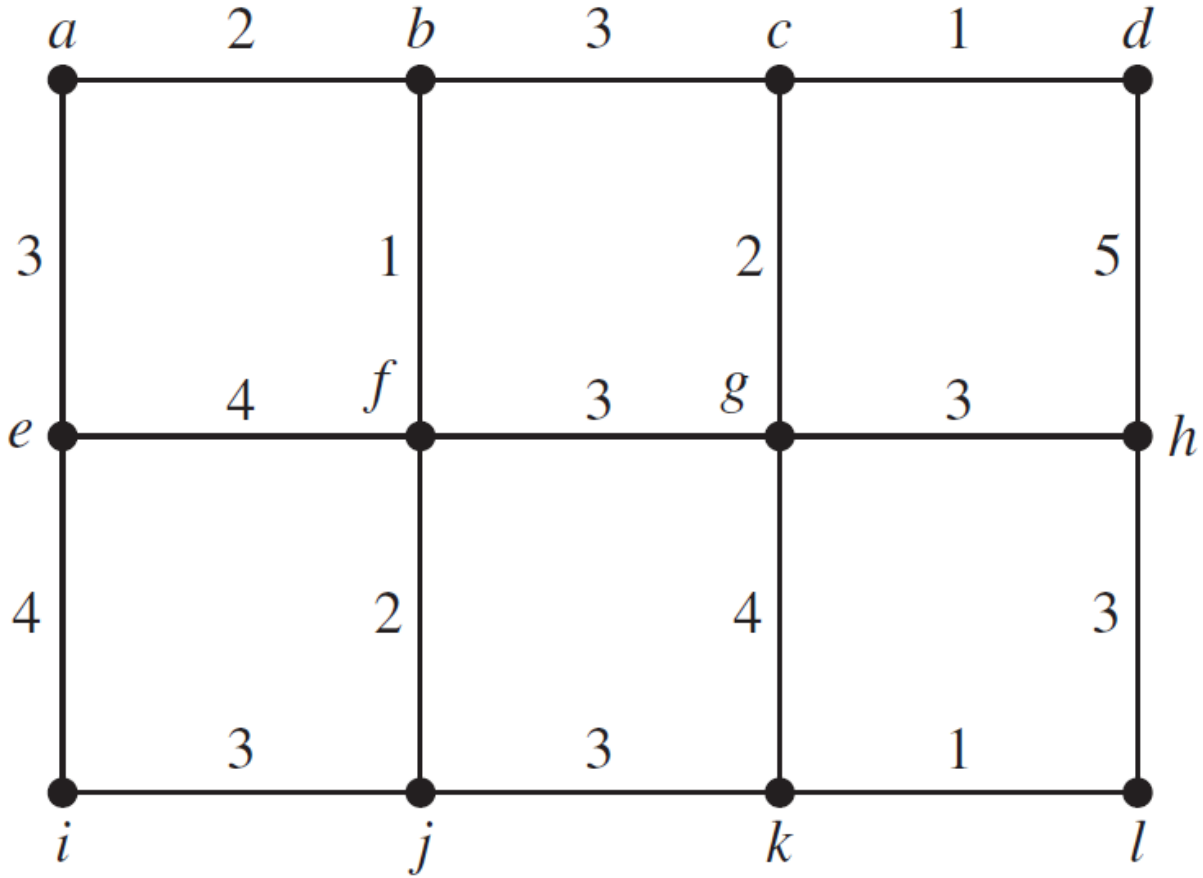
Examples

Find a minimum spanning tree using Kruskal's algorithm of the weighted graph



Answer: (Chicago, Atlanta), (Atlanta, New York) (Chicago, San Francisco), (San Francisco, Denver) with cost:700+800+1200+900=3600.

Find a minimum spanning tree using Kruskal's algorithm of the weighted graph. What is the minimal cost?



Answer: 24

Answer Q6

Complexities

to find a minimum spanning tree of a graph with m edges and n vertices

- using Prim's algorithm can be carried out using $O(m \log n)$ operations
- using Kruskal's algorithm can be carried out using $O(m \log m)$ operations

If $m < n(n-1)/2$ (the total number of possible edges in an undirected graph with n vertices), then it is more efficient to use Kruskal's algorithm.

Otherwise, there is little difference in the complexity of these two algorithms.

Homework:

- Solve odd exercises until #25 starting at p. 802.